

Scratch for Parents: Learn It, Then Teach It

Every quiz answered and explained, plus reference solutions for each lab — for all five modules. Use for checking work and reteaching. Confidential instructor material.

5 modules

Quiz answers + lab solutions

Confidential – do not distribute

CONTENTS

MOD 1 Getting Started with Scratch

- 1.1 What Is Scratch & Why Kids Love It
- 1.2 A Guided Tour of the Scratch Editor
- 1.3 Your First Project — Make the Cat Move & Talk

MOD 2 Core Coding Concepts

- 2.1 Sequences & Timing
- 2.2 Loops — Making Things Repeat
- 2.3 Conditionals & Sensing — Making Choices

MOD 3 Building Real Projects

- 3.1 An Interactive Animated Story
- 3.2 A Simple Game with Variables & Score
- 3.3 Debugging & Remixing — Fixing and Improving

MOD 4 Leveling Up — Bigger Projects

- 4.1 Clones — One Sprite, Many Copies
- 4.2 My Blocks — Make Your Own Blocks
- 4.3 Capstone Part 1 — Plan It & Build the Core
- 4.4 Capstone Part 2 — Lives, Game Over & Launch Day

MOD 5 Teaching Scratch to Your Child

- 5.1 The Art of Not Taking Over
- 5.2 Sharing Safely & the Scratch Community
- 5.3 ScratchJr — Coding for Ages 5–7

5.4 What Comes After Scratch

Module 1 — Answers & Solutions

Correct self-check answers with the "why", plus notes on each lab: what the reference project contains, the target scripts, and the common wrong turns and how to coach them. Q1–Q3 in each lesson come from the course's Quick Checks; Q4 is new to the kit.

Lessons 1.1–1.3

12 quiz items

3 labs

1.1 • WHAT IS SCRATCH & WHY KIDS LOVE IT

Q1

What is the single biggest advantage of block-based coding for children?

B) You can't make syntax/typing errors that break the program

Why: blocks only snap together when the combination is valid, so there's nothing to misspell. Speed (A) and cost (C) aren't what makes blocks different from typing.

Q2

In Scratch, what does a block's color tell you?

B) Which category it belongs to (Motion, Looks, Events...)

Why: blue is Motion, purple is Looks, gold is Events, and so on. The order comes from how blocks are stacked, not from their color.

Q3

What's the best first-session goal with your child?

C) Create one moment of "whoa, I made that!"

Why: that feeling is the hook everything else builds on. A finished game or memorized vocabulary puts too much pressure on day one.

Q4

In Scratch, the orange cat is an example of a...

B) Sprite

Why: a sprite is any character or object you can code. The stage is the screen it acts on, and blocks are the instructions. Anyone who picks A is usually mixing up "the thing" with "where the thing is", so point at each one on the projector.

LAB NOTES — CLICK AROUND (NO FILE)

Success = each participant has dragged at least one block into the code area, clicked it, and tried to snap a hat block *under* another block (it won't go). Coach the frozen participant by putting a mouse in their hand and saying "Drag anything. You can't break it." Check the Teach-It pitch for three things: it leads with the child's interest, uses "make" rather than "learn", and ends with a question.

1.2 · A GUIDED TOUR OF THE SCRATCH EDITOR

Q1

Nothing happens when your child runs their blocks. Most likely cause?

B) The code is on a different sprite than the one selected

Why: code lives on the selected sprite, so check the sprite list first. Scratch has no paid tier (A), and the internet (C) isn't the usual culprit.

Q2

Which button runs the project?

A) The green flag

Why: the flag starts every **when**  **clicked** script. The red stop halts everything; the Costumes tab switches to a sprite's outfits.

Q3

What does a block's category color help you do?

B) Quickly find the kind of block you need in the palette

Why: color groups blocks, and the colored dots jump the palette to each group. Color doesn't affect speed, and saving lives in the File menu.

Q4

You're not signed in. How do you keep today's project?

C) File → Save to your computer

Why: this downloads a `.sb3` file you can reopen with File → Load from your computer. Accounts are optional (A is wrong), and only signed-in projects save to the cloud (B is wrong).

LAB NOTES — POINT AND NAME + SAVE & RELOAD (NO FILE)

Correct labels: **1 Blocks Palette** (left), **2 Code Area** (middle), **3 Stage** (upper right), **4 Sprite List** (lower right). The default cat has 2 costumes. The category menu shows 9 colored dots (Motion, Looks, Sound, Events, Control, Sensing, Operators, Variables, My Blocks), plus more if extensions are added. The program-starter block is **when**  **clicked** in gold Events. Save is done when the participant has reopened the `.sb3` with Load from your computer, or sees their project under My Stuff.

Common wrong turns: swapping "stage" and "sprite list" in the sketch (have them click a sprite in the list and watch the code area change); a saved file they can't find (open Downloads together); calling the Code Area "the stage".

1.3 · YOUR FIRST PROJECT: MAKE THE CAT MOVE & TALK

Q1

Which block goes at the top so the script runs on the green flag?

B) when  clicked

Why: the "hat" block starts the program. **move** and **say** are action blocks that go underneath it.

Q2

Scratch runs the blocks in a stack...

A) from top to bottom, in order

Why: this is a **sequence**, the most basic idea in coding. Order is behavior.

Q3

The cat moves but won't talk. Best first check?

B) Is the "say" block snapped onto the stack?

Why: a block sitting loose, even a few pixels away, doesn't run with the stack. Drag the hat: anything that doesn't come along isn't attached.

Q4

say ... for 2 seconds is dragged above move 50 steps. What happens?

C) The cat talks first, then walks

Why: the timed **say** block finishes its 2 seconds before the next block starts, so the steps happen one after another. B is a good wrong answer to discuss, because running things "at the same time" needs two scripts (Module 2).

LAB SOLUTION — MAKE THE CAT MOVE & TALK

Reference: `labs/module-1/l1-3-cat-move-talk.sb3`. One sprite (the default cat, Sprite1) on the default white backdrop, with the lesson's finished script. Clicking the green flag makes the cat walk 50 steps to the right and then say its message for 2 seconds.

The image displays two Scratch scripts side-by-side. The left script consists of three blocks: a yellow 'when green flag clicked' block, a blue 'move 50 steps' block, and a purple 'say Hello! I'm Scratch Cat for 2 seconds' block. The right script consists of six blocks: a yellow 'when green flag clicked' block, a blue 'go to x: 0 y: 0' block, a blue 'move 50 steps' block, a purple 'next costume' block, a purple 'start sound Meow' block, and a purple 'say Hello! I'm Scratch Cat for 2 seconds' block.

Accept any version where the hat is on top, move and say are snapped under it, and the message is the participant's own. "Make it yours" additions (a second **say**, **change size by 25**, **turn 15 degrees**) are all fine anywhere under the hat.

COMMON WRONG ANSWERS & HOW TO COACH THEM

Floating blocks (they look attached but aren't). Coach: "Grab the gold block and drag it. What came with it?"
Let them find the gap themselves.

Script built on the stage/backdrop instead of the cat (the Stage has no Motion blocks, so they'll be confused about where "move" went). Coach: "Which picture is highlighted in the sprite list?"

No hat block. Clicking the stack still runs it, so they think it works, but the green flag does nothing. Coach: "Press the flag. What's missing on top?"

Plain *say Hello!* (no seconds). The bubble never goes away. That's not wrong, just different. Ask which one they meant to use.

Cat stuck at the right edge after several runs. Coach toward **go to x: 0 y: 0**: "Where should the cat be at the *start* of every run?"

Upside-down cat after **turn ↻ 15 degrees**. Set the rotation style to left-right in the direction pop-up, or add **point in direction 90** after the hat.

Module 2 — Answers & Solutions

Correct self-check answers with the "why," plus what each reference lab project contains and how to coach the usual wrong turns. Questions 1–3 of each lesson come from the course's Quick Checks. Q4 is new to the kit.

Lessons 2.1–2.3

12 quiz items

3 Labs

2.1 • SEQUENCES & TIMING

Q1

What is a "sequence" in coding?

A) Steps carried out in order, one after another

Why: steps run top to bottom, exactly as stacked. B is wrong because order is exact, not random. C describes a loop (Lesson 2.2).

Q2

Which category holds the wait block?

B) Control (orange)

Why: *wait (1) seconds* is the first block in Control. Blue is Motion; purple is Looks.

Q3

Your child's cat "freezes" for two seconds mid-routine. Most likely reason?

B) A "say for 2 seconds" or "wait" block is holding, as designed

Why: the block is doing its job. Point at the words "for 2 seconds." Adults who pick A are voicing the fear of breaking the computer. Reassure them and move on.

Q4

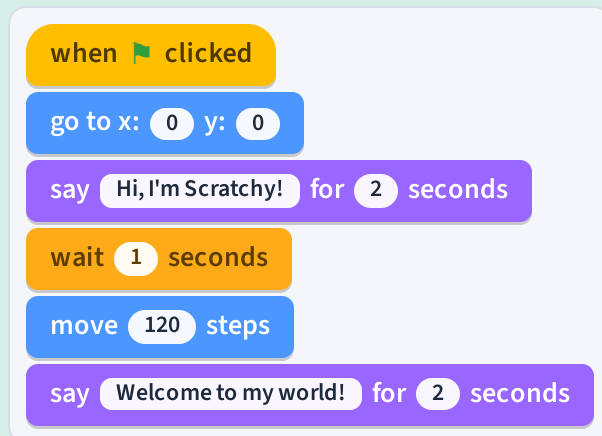
say Hello! for 2 seconds vs plain say Hello!?

B) The timed one pauses the script before the next block; plain say moves straight on

Why: the timed version shows the bubble, waits, removes it, then continues. Plain *say* leaves the bubble up and runs the next block at once. For step-by-step scenes, use the timed one.

LAB SOLUTION — GREETING ROUTINE

Reference: `labs/module-2/l2-1-greeting-routine.sb3` . One script on the cat sprite:



Challenge version: *start sound (Meow)* and *next costume* go between the wait and the move.

Common wrong turns & coaching: (1) *No go to x: 0 y: 0.* The cat starts wherever the last run left it and eventually walks off-stage. Ask: "Where does the cat start the second time?" (2) *Plain say instead of say-for-seconds.* Both bubbles flash by, or the first is instantly replaced. Ask them to read the two block labels aloud. (3) *A block not snapped.* It sits a few pixels away and never runs. Drag the stack by its hat block; anything left behind isn't attached. (4) *Two green-flag scripts.* An old S1 script also runs, so the cat does two things at once. Right-click the code area → Clean up Blocks to find strays.

2.2 · LOOPS — MAKING THINGS REPEAT

Q1

You want the cat to blink exactly 5 times. Best loop?

A) repeat 5

Why: you know the count. *forever* would never stop, and *wait 5 seconds* is a pause, not a loop.

Q2

Only which blocks repeat in a loop?

B) Only the blocks inside the C-shape

Why: blocks nested in the loop's "mouth" repeat. Blocks above or below it run once. Block color has nothing to do with it.

Q3

A block placed after a "forever" loop will...

A) never run

Why: the program never gets past *forever*. Scratch hints at this: *forever* has no notch on its bottom, so nothing can snap below it.

Q4

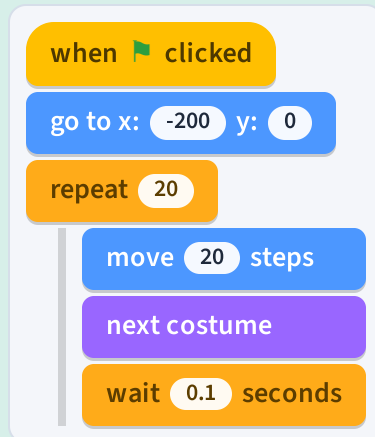
The walking cat zooms across in a blink. Best first fix?

A) Add (or lengthen) a wait inside the loop

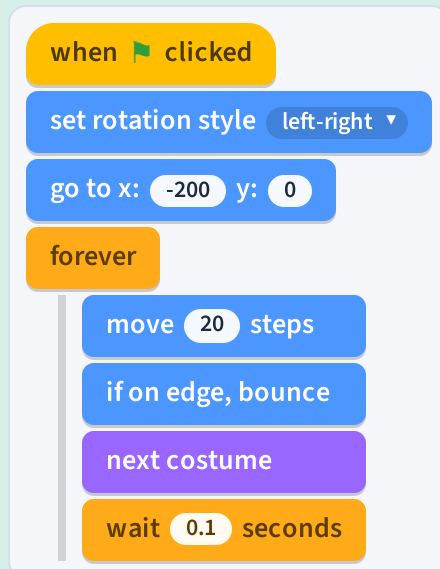
Why: the wait has to repeat with each step to slow every step. A wait after the loop only pauses once, at the end. Try 0.1–0.3.

LAB SOLUTION — A REAL WALKING CAT

Reference: `labs/module-2/l2-2-walking-cat.sb3` . One script on the cat sprite (which has two costumes):



Challenge (patrol) version. Wrapping the original repeat inside a forever works too; this is the tidier form:



Common wrong turns & coaching: (1) *next costume below the loop*. The cat glides with frozen legs. Ask: "Is the costume change inside the mouth, or outside?" (2) *No wait inside*. The cat teleports across. (3) *Upside-down cat* in the challenge. Add *set rotation style [left-right]*. (4) *A second forever script* keeps running after the challenge. The red stop ends it. Delete the leftover.

2.3 · CONDITIONALS & SENSING — MAKING CHOICES

Q1

What shape of block fits in an "if ___ then" slot?

A) A hexagon (true/false) block

Why: the slot is pointy and takes only true/false conditions, such as *touching (edge)?* or *key (space) pressed?*. Oval reporters give values and won't fit. Hat blocks only start scripts.

Q2

Why do most games wrap their "if" checks in a forever loop?

B) So the condition is checked constantly, not just once

Why: without the loop Scratch checks once, when the script starts, and never again. Tie it back to the warm-up: you check for rain all day, not once.

Q3

"if/then/else" is best when...

B) you want one thing if true and a different thing otherwise

Why: use else when the outcomes are opposites of the same question. The four arrow keys are separate questions, so they get four separate ifs.

Q4

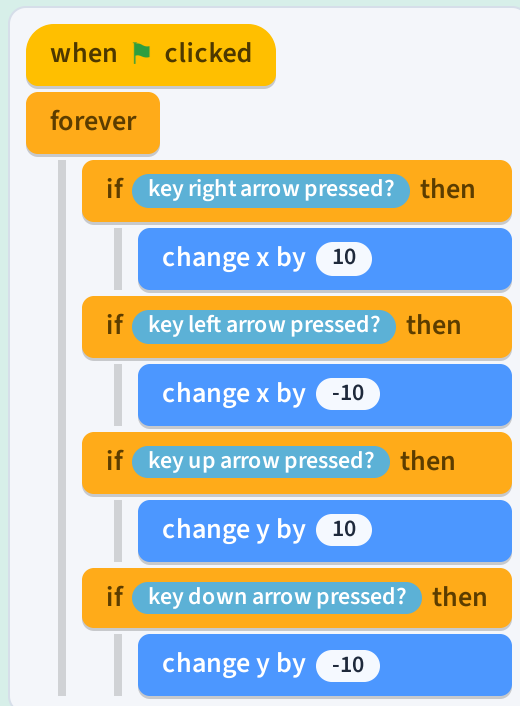
Script matches the example but the cat won't move. Try first?

A) Click the stage once so it has keyboard focus

Why: key presses go to whatever part of the page was clicked last. Click the stage, then check the nesting (change → inside if → inside forever). Restarting is never the first move.

LAB SOLUTION — DRIVE THE CAT WITH ARROW KEYS

Reference: `labs/module-2/12-3-arrow-key-cat.sb3` . One script on the cat sprite:



Challenge: a fifth if inside the same forever, *if <touching (edge)?> then say (Ouch!) for (1) seconds*. (While the bubble shows, the other keys pause, which is fine at this stage.)

Common wrong turns & coaching: (1) *if outside the forever*. The keys work for a split second at the start, then never again. Ask: "How often does it check?" (2) *x and y swapped*. Up moves the cat sideways. Say: "x is across, like a crossword; y is up and down." (3) *Wrong sign*. Left and down need -10 . (4) *when [right arrow] key pressed* hat blocks instead of the loop. That also works but moves jerkily because of the keyboard-repeat delay. Accept it, then show the smoother loop version. (5) *No keyboard focus*. Click the stage.

Module 3 — Answers & Solutions

Correct self-check answers with the "why," reference notes for each lab with the scripts drawn out, the full Bug Hunt solution (all four planted bugs), and coaching moves for the common wrong answers.

Lessons 3.1–3.3

12 quiz items

3 labs

4 planted bugs

3.1 • AN INTERACTIVE ANIMATED STORY

Q1

Two sprites need to take turns talking. What coordinates them?

A) broadcast and "when I receive"

Why: the next line starts only when the cue is actually sent. Matched `wait` times drift and break.

Q2

Which block moves a sprite smoothly over time?

B) glide _ secs to x/y

Why: `go to x/y` teleports instantly. `glide` slides over the number of seconds you give it.

Q3

Both characters talk at the exact same time. Likely cause?

A) Both start with "when  clicked" instead of "when I receive"

Why: every green-flag script starts at the same moment. Only line 1 should use the flag.

Q4

Where does the backdrop-changing script go?

B) On the Stage: "when I receive night" then "switch backdrop to"

Why: the Stage owns the backdrops and can listen for broadcasts just like a sprite. Click the Stage (right of the sprite list) before adding its blocks.

LAB SOLUTION — A CONVERSATION THAT PLAYS ITSELF

Reference: `labs/module-3/l3-1-conversation.sb3` . A Forest backdrop, a Cat and a Bear, and a chain of four cues: Cat on the flag → line 2 → Bear → line 3 → Cat → line 4 → both glide off. Words, characters, and coordinates will vary. Check the *structure*.

The image displays four Scratch scripts arranged in a 2x2 grid, each within a light purple rounded rectangle. The scripts are as follows:

- Top Left Script:**
 - when green flag clicked
 - go to x: -200 y: -100
 - glide 2 secs to x: -60 y: -100
 - say Hi Bear! for 2 seconds
 - broadcast line 2
- Top Right Script:**
 - when I receive line 2
 - say Hello Cat! for 2 seconds
 - broadcast line 3
- Bottom Left Script:**
 - when I receive line 3
 - say Want to explore? for 2 seconds
 - broadcast line 4
- Bottom Right Script:**
 - when I receive line 4
 - say Let's go! for 1 seconds
 - glide 2 secs to x: 240 y: -100

(The last script goes on *both* sprites, so they leave together. The Bear's version usually skips the `say` .)

Common wrong answers and how to coach them: both sprites talking at once → "Which of your scripts start with the flag? Only line 1 should." · a line that never plays → the `when I receive` dropdown names a different message ("line2" vs "line 2"). Open the dropdown and compare. · the line-4 glide only moves one sprite → the receiving script was added to one sprite only. · the Bear has no Bear scripts → they were built while the Cat was selected. Drag the script onto the Bear in the sprite list to copy it.

3.2 · A SIMPLE GAME WITH VARIABLES & SCORE

Q1

What is a variable?

A) A named box that stores a value the program can change

Why: it has a name (`score`) and a value (0, then 1, then 2...), like a whiteboard labeled SCORE.

Q2

To add one point to the score, you use...

B) change score by 1

Why: `change` adds to what's already there. `set score to 1` would reset it to 1 every time.

Q3

Where should "set score to 0" go?

A) Right after the green flag, before the game loop

Why: it should run once per game. Inside the `forever` it would wipe the score on every drop.

Q4

You can't see the score on the stage. What do you do?

C) Tick the checkbox next to the variable in the Variables palette

Why: the checkbox toggles the live readout on the stage. (The `show variable` block does the same thing from code.)

LAB SOLUTION — CATCH THE APPLE

Reference: `labs/module-3/l3-2-catch-the-apple.sb3` . Two sprites (Cat, Apple) and one variable, `score` , set "For all sprites" with its stage readout showing. This file is also the **solution to the Bug Hunt**.

The image displays two Scratch code snippets. The left snippet is a 'when green flag clicked' event handler. It sets the 'score' variable to 0 and enters a 'forever' loop. Inside the loop, it moves the Apple sprite to a random x-coordinate between -200 and 200 and y-coordinate 170. It then enters a 'repeat until touching Cat?' loop, which moves the Apple down by 8 pixels. If the Apple's y-position is less than -170, it moves it to a new random position. Finally, it increments the score by 1. The right snippet is a 'when green flag clicked' event handler that enters a 'forever' loop, moving the Cat sprite to the mouse's x-coordinate and y-coordinate -140.

Common wrong answers and how to coach them: score jumps by several per catch → the apple lingers on the cat. Make sure the loop goes back to the top `go to` right after scoring. · score never rises → the `touching` dropdown says "mouse-pointer," or the Cat's y keeps it below the apple's path. · apple frozen at the top → they used `if touching edge?` for the bottom check. The apple starts touching the *top* edge, so it resets every frame. Use `y position < -170` . · "mouse x" typed as text → it must be the round Sensing reporter dropped into the slot. · *Timer challenge*: a second green-flag script, `set [time left] to 30` → `repeat until <(time left) = 0>` → `wait 1 seconds` , `change [time left] by -1` → then `broadcast [game over]` , `stop all` .

3.3 · DEBUGGING & REMIXING

Q1

The golden rule of debugging is...

A) Change one thing at a time, then test

Why: change five things and you'll never know which one mattered, or which one broke something new.

Q2

What does "See Inside" do on a shared project?

A) Opens the project's code to explore how it was made

Why: it turns the whole Scratch community into a free textbook. **Remix** (signed in) is the separate step that makes your own copy.

Q3

Your child is melting down over a stubborn bug. Best move?

B) Have them explain it aloud, or take a short break

Why: the rubber duck often surfaces the bug mid-sentence, and a break is a real debugging strategy. Grabbing the mouse ends the learning.

Q4

Green flag clicked and nothing happens. Most likely suspect?

B) The code is on a different sprite than you think

Why: it's the first row of the usual-suspects table. Click each sprite in the sprite list and look for the script.

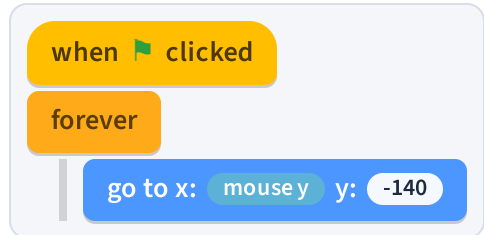
LAB SOLUTION — THE BUG HUNT

Starter: `labs/module-3/l3-3-bug-hunt-starter.sb3` is the 3.2 game with four planted bugs. Solution: `labs/module-3/l3-2-catch-the-apple.sb3` (scripts drawn above). Each bug below shows the **symptom** a pair should *watch*, the **cause**, and the **fix** with the corrected script. The symptoms overlap (with bug 4 in place, the score races even while bug 3 keeps the apple stuck at the top), so accept any order, as long as each fix was one change followed by a test.

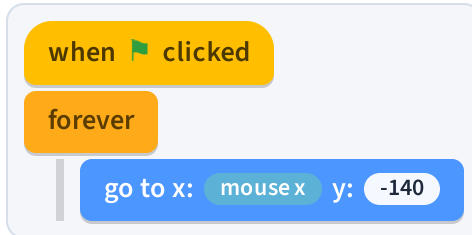
Bug 1 · Cat script

Symptom: the cat ignores left-right mouse movement and slides sideways when the mouse moves *up and down*. It "drifts strangely."

Planted (wrong)



Fixed



Fix: replace the `mouse y` reporter with `mouse x` (Sensing). The x slot needs the mouse's left-right number.

Coach: "Move the mouse only sideways. Does the cat? Which number does the x slot get?"

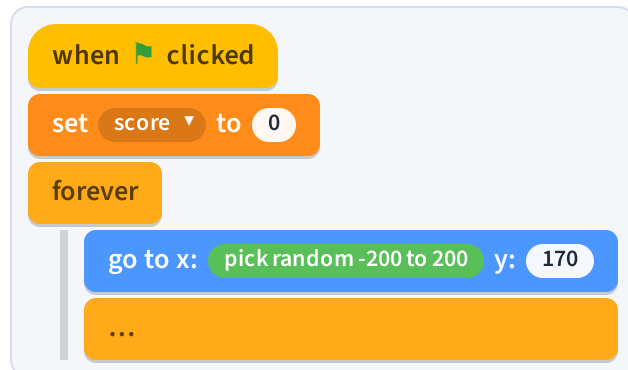
Bug 2 · Apple script, top

Symptom: the score doesn't reset. Press the green flag again and it carries on from the last game's number. (The first game may even open with a leftover score.)

Planted (wrong)



Fixed

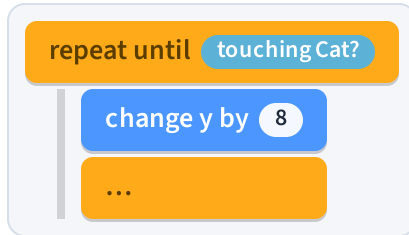


Fix: add `set [score] to 0` between the hat block and `forever`. **Common wrong fix:** putting it *inside* the forever, which zeroes the score on every drop so it never climbs past 1. That's a good teachable second bug. **Coach:** "What wipes the whiteboard at the start of a game?"

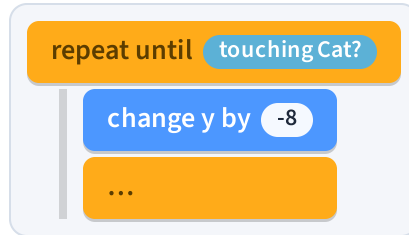
Bug 3 · Apple script, falling loop

Symptom: the apple never falls. It sits at the top of the stage, so nothing can ever be caught. (While bug 4 is still in place, the score races up even though the apple isn't moving. That clue points at the loop.)

Planted (wrong)



Fixed



Fix: change the **change y by** value from **8** to **-8**. Bigger y means *up*, so the apple keeps pushing into the top of the stage (Scratch holds it on-screen) instead of falling. **Helper to suggest:** tick the Apple's **y position** checkbox (bottom of Motion) and watch. The number never goes down. **Coach:** "Which way does y go when it gets bigger? Which block moves the apple up or down?"

Bug 4 · Apple script, nesting

Symptom: the score races upward *while the apple is still falling*, one point per step, whether or not it's caught.

Planted (wrong)



Fixed



Fix: drag `change [score] by 1` out of the repeat-until's mouth and drop it directly *below* the repeat-until (still inside `forever`). It should run once, after the catch. **Common wrong fix:** dropping it loose on the canvas or trying to attach it under `forever` (which has no bottom notch). A block that isn't connected never runs. Have them drag the script and check that the +1 moves with it. **Coach:** "How many times does the +1 run while one apple falls? Which loop is it inside?"

COACHING THE BUG HUNT AS A WHOLE

Pairs that fix everything by rebuilding from scratch: accept it, then ask them to name the one block that was different. **Pairs that fix bug 3 and think the score is still broken:** it is. That's bug 4. Praise the one-change discipline that let them tell the two apart. **Pairs who rewrite the bottom check as `if touching edge?`:** the apple freezes at the top again, because at y: 170 it already touches the *top* edge. That's a bonus teachable moment, and it's why the reference uses `y position < -170`. **Fully fixed check:** the cat follows the mouse sideways, the apple falls, a miss scores nothing and restarts at the top, a catch scores exactly 1, and a second green flag starts again at 0.

Module 4 — Answers & Solutions

Correct self-check answers with the "why," plus what each reference lab file contains, the scripts drawn out, and the common wrong turns with how to coach them. Questions 1–3 in each lesson come from the course's Quick Checks; Q4 is kit-only.

Lessons 4.1–4.4

16 quiz items

4 lab files

4.1 • CLONES — ONE SPRITE, MANY COPIES

Q1

The original star is hidden. You make clones, but none appear. Why?

B) Clones start hidden, just like the original, and need a "show" block

Why: a clone copies everything about the original, including being hidden. Add **show** right under **when I start as a clone**.

Q2

Which script runs only on the new clones, never on the original?

C) when I start as a clone

Why: **when clicked** runs only on the original (clones are made after the click); **when I receive** runs on the original *and* every clone.

Q3

The game slows, then new stars stop appearing. Most likely cause?

A) Clones are never deleted, so they pile up toward the 300 limit

Why: off-screen clones still count. Past about 300, **create clone** silently does nothing. Every clone must reach **delete this clone**.

Q4

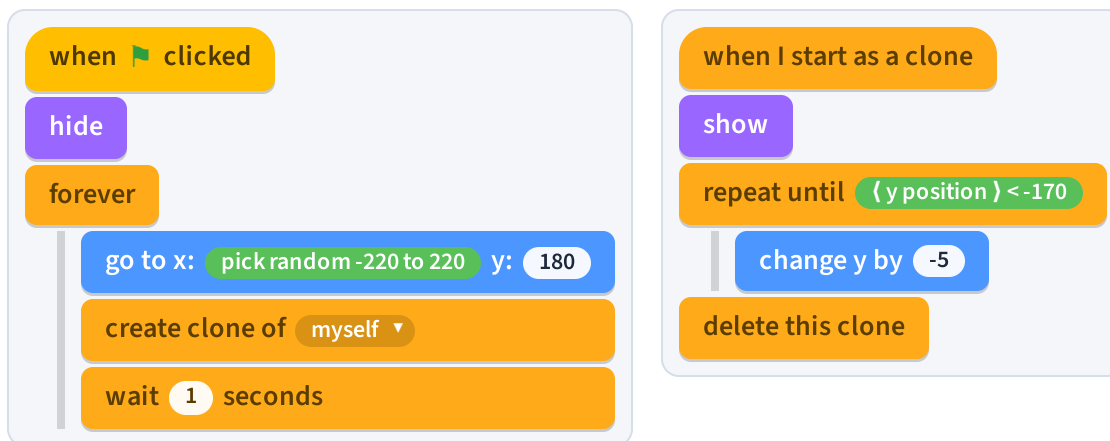
create clone of myself inside the when I start as a clone script — what happens?

B) Every clone makes another clone, flooding the screen until Scratch's limit

Why: each new clone runs the clone script, so each one stamps again: a chain reaction. Coach with "Who should do the stamping — the stamp or the prints?"

LAB SOLUTION — FALLING STARS · LABS/MODULE-4/L4-1-FALLING-STARS.SB3

One **Star** sprite on a dark backdrop, with two scripts. Optional extras (random size, color effect, spin) are not needed for credit.



◆ COMMON WRONG ANSWERS & HOW TO COACH THEM

- ◆ **Nothing ever appears** → no **show** in the clone script. Ask: "The stamp is hidden. What do its prints look like?"
- ◆ **One clump of stars** → no **go to** before stamping, or no **wait**. Ask: "Where is the stamp when it presses down? Does it ever move?"
- ◆ **Stars vanish instantly** → **repeat until touching edge?**. Each star is born touching the top edge. Ask: "Where does a new star start?"
- ◆ **The original falls** → the falling loop is under **when green flag clicked**. Ask: "Is this running on the stamp, the prints, or both?"
- ◆ **y position in the wrong slot** (right side of <) → the test reads "-170 < y" and is true at once. Read the block aloud together, left to right.

4.2 · MY BLOCKS — MAKE YOUR OWN BLOCKS

Q1

Where do you put the steps your new block should do?

B) Under the pink "define" block

Why: the define hat is the recipe card. The green-flag script only *calls* the block; the dialog only sets name and inputs.

Q2

The jump works but the cat teleports instead of hopping. Likely cause?

A) "Run without screen refresh" is ticked

Why: with refresh off, Scratch skips drawing the in-between frames. Right-click the define hat → Edit → untick.

Q3

reset game was made on the Bowl but is missing on the Star. Why?

C) Custom blocks belong to the sprite they were made in

Why: My Blocks are per sprite. Drag the define script onto the other sprite's thumbnail in the sprite list to copy it.

Q4

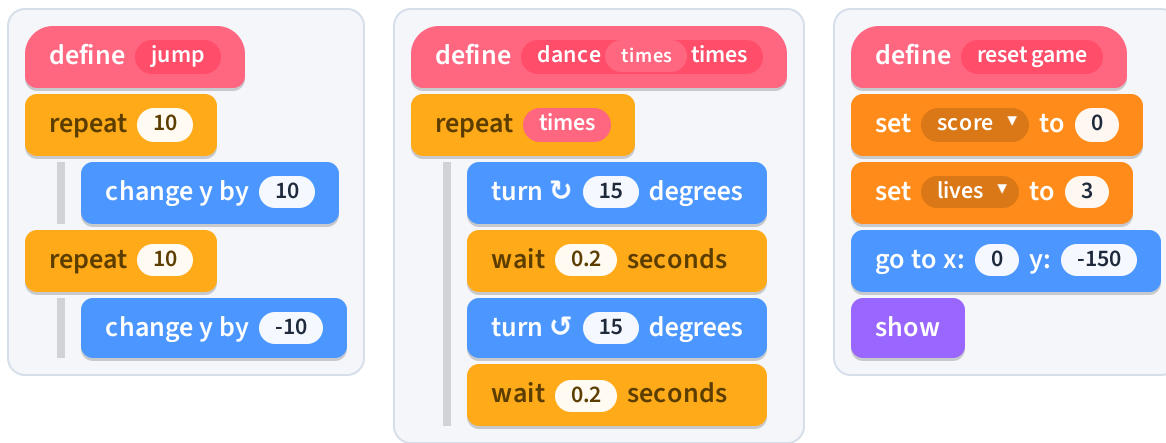
How do you make the repeat use the number passed to dance (times)?

B) Drag the pink times oval out of the define hat into the repeat slot

Why: dragging the oval gives you a copy that holds the passed-in value. Typing "times" is just text; a new variable is a different thing and won't receive the number.

LAB SOLUTION — THREE CUSTOM BLOCKS · LABS/MODULE-4/L4-2-MY-BLOCKS.SB3

The cat sprite with the **jump** block called from **when space key pressed** and **when this sprite clicked**; **dance (times)** times called with 3 and 6; variables **score** and **lives** (For all sprites) and a **reset game** block called from the green flag. The **jump (height)** challenge is optional.



◆ COMMON WRONG ANSWERS & HOW TO COACH THEM

- ◆ **"My block does nothing"** → steps sitting near, but not snapped under, the define hat. Drag the hat: whatever moves with it is attached.
- ◆ **Dance never repeats / repeats 0 times** → the word typed into the slot. Point at the pink oval: "The value lives in here — drag it out."
- ◆ **"My block disappeared"** → a different sprite is selected. Click the sprite it was made in.
- ◆ **reset game shows a red error / empty dropdown** → variables not made yet. Make **score** and **lives** first.

4.3 · CAPSTONE PART 1 — PLAN IT & BUILD THE CORE

Q1

A huge idea with dragons, shops, and ten levels. Best first move?

B) Pick the ONE main action, build that small version first, and save the rest for a "someday list"

Why: games started by dragging blocks grow too big to finish. Shrink where you start without crushing the idea.

Q2

Why does the original Star hide and only make clones?

A) It's a hidden "factory." Only the clones fall and get caught

Why: hidden sprites can still make clones; that's the whole job of the original. Hiding doesn't affect speed.

Q3

Why hide and play sound Pop until done before delete this clone?

B) Deleting a clone cuts off its sound, so it hides and finishes the pop first

Why: **hide** makes the star vanish instantly (it feels caught); **until done** lets the pop finish before the clone goes.

Q4

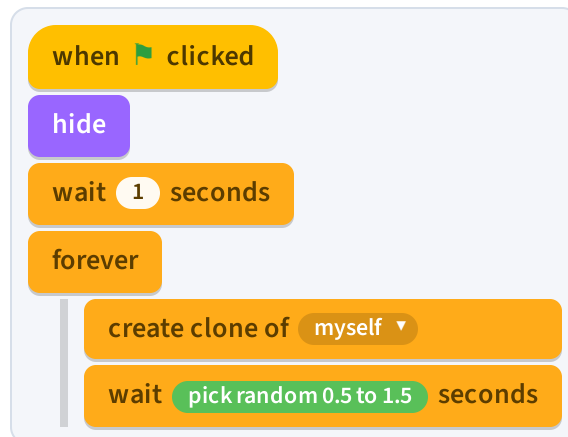
Which is the "smallest playable version" of Star Catcher?

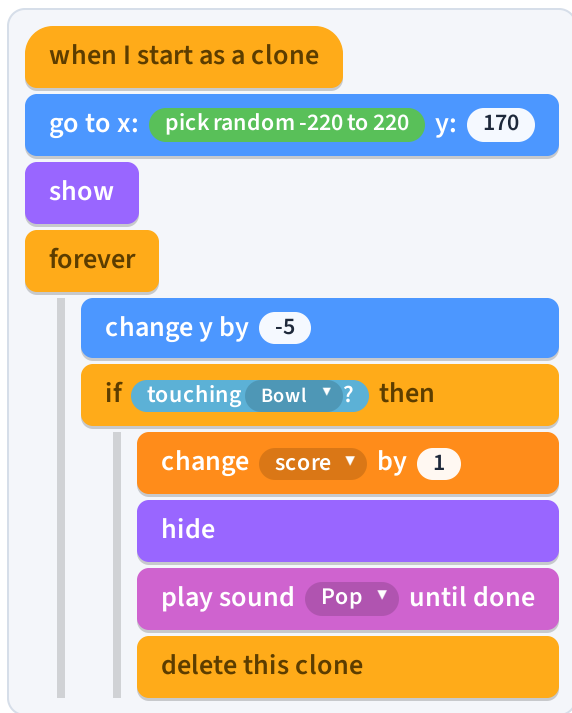
A) Bowl moves + stars fall + catching adds score

Why: it's the least you can build that is still a game someone can play. Lives and GAME OVER are additions (Part 2); the card alone isn't playable.

LAB SOLUTION — STAR CATCHER PART 1 · LABS/MODULE-4/L4-3-STAR-CATCHER-PART1.SB3 (ALSO THE S5 RESCUE FILE)

End of step 7. Sprites: **Bowl** and **Star** (the cat deleted) on the **Stars** backdrop. Variables **score** and **lives**, For all sprites, shown on stage. The Bowl holds **define reset game** (as in 4.2) and the movement script; the Star holds the factory and the clone script, with the **Pop** sound added. Missed stars still pile up at the bottom: that is correct for Part 1.





◆ COMMON WRONG TURNS & HOW TO COACH THEM (COACH THE PARENT; LET THE CHILD FIX)

- ◆ **Bowl won't move** → stage lacks keyboard focus, or the **ifs** sit outside the **forever**. "Click the stage once." Then trace the **forever's** mouth with a finger.
- ◆ **A frozen star at the top** → the original isn't hidden. "Which star is the factory? Should we see it?"
- ◆ **Score doesn't reset on replay** → **reset game** not called, or placed inside the **forever** (score stuck at 0).
- ◆ **Pop is cut off** → **start sound** used, or **delete this clone** before the sound.
- ◆ **lives made "For this sprite only"** → looks fine now but breaks step 8. Check it at the checkpoint: a sprite-only variable's stage readout shows the sprite name first (**Bowl: lives**). Easiest fix: delete it and remake it For all sprites.
- ◆ **Design card (not graded)**: look for one main action, a clear way to score and to lose, and a Someday List with at least one item. If a card lists five main actions, help the family circle one.

4.4 · CAPSTONE PART 2 — LIVES, GAME OVER & LAUNCH DAY

Q1

Each missed star takes 2 or 3 lives instead of 1. Likely cause?

B) delete this clone isn't inside the lives if, so the star keeps subtracting

Why: the star stays below -170 and loses a life on every trip around the forever until it's deleted. Delete must be the last block inside that **if**.

Q2

Why does the Game Over sprite show itself before stop all?

A) So the sign is already on screen when everything freezes

Why: nothing runs after **stop all**. It also removes every clone, which is what we want.

Q3

The game breaks near the finish; your child is close to tears. Best move?

B) Point out it worked before the last change, look at just that part together, or save and stop for today

Why: shrinking the bug to the last change makes it solvable, and stopping to come back fresh is a real programmer move. Taking the mouse takes the win away.

Q4

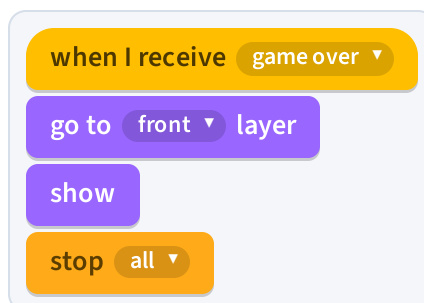
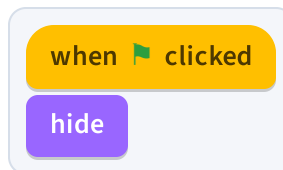
GAME OVER is still on screen when a new game starts. The fix?

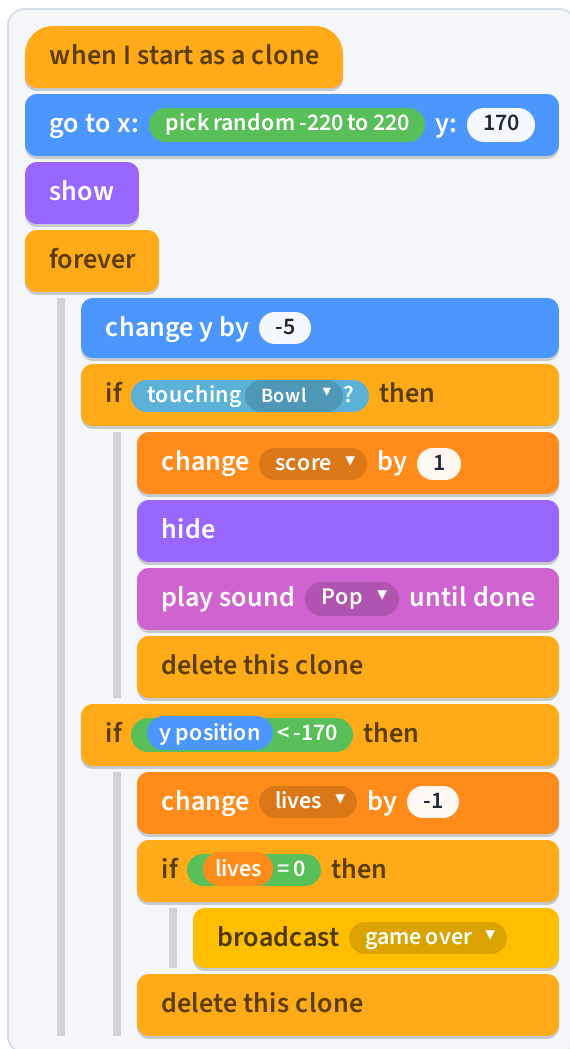
C) Give the Game Over sprite a "when  clicked → hide" script

Why: the sign stays shown after **stop all** until something hides it. Moving **stop all** first would stop the **show**; lives at 0 would end every game instantly.

LAB SOLUTION — STAR CATCHER FINAL · LABS/MODULE-4/L4-4-STAR-CATCHER-FINAL.SB3

Everything in Part 1, plus: the lives **if** in the Star's clone script, a **game over** message, a painted **Game Over** sprite with the two scripts below, and a Stage music loop (**when  clicked → forever → play sound ... until done**). It meets every line of the Project Completion checklist. Someday-list extras (speed-up, rocks, high score, levels) are bonuses, not requirements.





◆ COMMON WRONG TURNS & HOW TO COACH THEM

- ◆ **Stars still pile up** → the **y position** check is outside the forever (it runs once, at birth). "Does the star ask this question once or all the way down?"
- ◆ **Lives drop by more than 1** → **delete this clone** outside the lives if (Q1).
- ◆ **GAME OVER never shows** → broadcast and receive use different messages, or lives never hit exactly 0 (not reset to 3, or a "For this sprite only" lives on the Bowl that the Star can't see).
- ◆ **Music stutters or overlaps** → **start sound** in the forever. Use **play sound ... until done**.
- ◆ **Ages 5–7 skipped lives** → that is a complete, correct outcome. Credit the painted sign, the music, and the launch.
- ◆ **Launch day:** "correct" means the child explained the rules, kept hands off while someone else played, and collected one idea for the Someday List.

Module 5 — Answers & Solutions

Correct self-check answers with the "why," plus what good looks like in each lab. Module 5 has no reference `.sb3` files: its labs are role-plays, a safety checklist, a ScratchJr build, and a next-project plan. The look-fors below double as an observer checklist for trios.

Lessons 5.1–5.4

16 quiz items

4 labs + showcase

5.1 • THE ART OF NOT TAKING OVER

Q1

What is "scaffolding" in teaching?

A) Giving just enough help, then fading it as they gain confidence

Why: like training wheels raised bit by bit. B turns the child into a spectator; C leaves them stalled.

Q2

Which praise best builds resilience?

B) "You kept trying different things until it worked."

Why: it names a strategy the child can repeat. "You're so smart" makes the next bug feel like proof of the opposite; "that was easy" dismisses their effort.

Q3

Your child is bored because the task is too easy. Best response?

A) Offer a new challenge ("What if it kept score?")

Why: bored = raise the bar back into productive struggle. Simplifying (B) makes boredom worse.

Q4

"Just do it for me!" Best response?

B) Offer to be their "hands" while they're the "boss"

Why: the lesson's own answer. The child keeps all the thinking; within a session or two most kids want the mouse back. A robs the learning moment; C is "too little help."

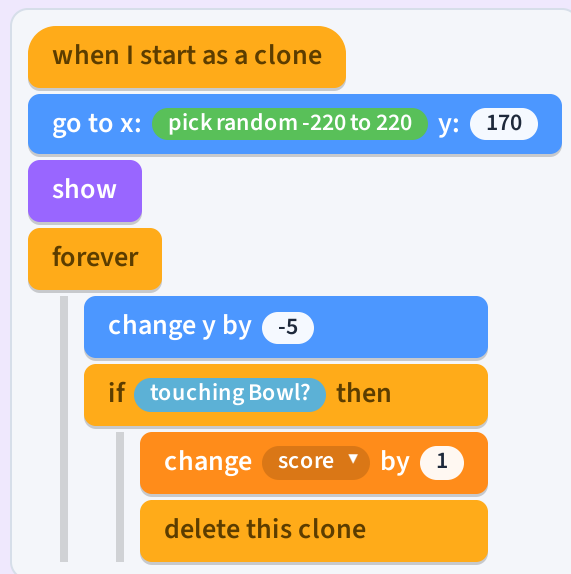
LAB — HANDS-OFF COACHING ROUNDS: WHAT GOOD LOOKS LIKE

The prop is `labs/module-3/l3-3-bug-hunt-starter.sb3` (four planted bugs; the fixed version is `l3-2-catch-the-apple.sb3`) or any half-finished Star Catcher. Participants don't need to fix every bug; the goal is the coaching, not the project.

Look-fors: coach's hands never touch the mouse or trackpad · at least three toolkit questions · a visible pause before speaking · at least one process-praise line · a "yet" · coach shrinks the step when the "child" escalates · the "child" does every drag.

WORKED-EXAMPLE BUG (WORKBOOK) — THE FIX

The star's *if touching Bowl?* check sat above the *forever*, so it ran once, at the top of the screen. It must live inside the loop:



COMMON WRONG MOVES & HOW TO COACH THEM

The disguised answer ("Don't you think it should go inside the forever?") → "That's the answer with a question mark. Try: *when does it check?*"

The walk-away ("Figure it out") → too little help; move one rung down the ladder, don't leave.

Talent praise on success ("You're a genius!") → swap for what they *did*: "You read the script line by line."

Never shrinking the step while the "child" gets louder → "What's the tiniest thing that would be a win right now?"

5.2 · SHARING SAFELY & THE SCRATCH COMMUNITY

Q1

When is a child's Scratch project visible to the public?

B) Only after they click "Share"

Why: projects are private by default; saving (C) keeps them in the account, not public.

Q2

Which is TRUE about the Scratch community?

A) It's moderated, has guidelines, and has no private DMs

Why: interactions are public and visible to you, and every comment can be reported, deleted, or turned off.

Q3

Best safety habit for a young sharer?

B) A non-identifying username, your email on the account, and regular check-ins

Why: hides real identity, keeps you in the loop on account notices, and makes "show me what you made" a normal habit.

Q4

Your child gets an unkind comment. Best first response?

C) Read it together, report it, delete it, and talk about how it felt

Why: the lesson's "team job." Consider turning comments off on that project for a while. Deleting the account (A) punishes the child; replying (B) feeds the conflict.

LAB — SAFETY CHECKLIST & FAMILY RULES: WHAT GOOD LOOKS LIKE

A finished `labs/module-5/safety-checklist.html` plus 3–5 family rules that cover: **what never gets shared** (real name, address, school, phone, photos) · **where Scratch is used** (shared device / common room) · **what to do with a mean comment** (show a grown-up; read, report, delete together) · ideally **credit when remixing**. Rules are written *with* the child at home and signed by both.

Scenario role-play look-fors: the coach says yes to the child's excitement before setting guard-rails · proposes a non-identifying username · uses the "before sharing" script or their own version of it · treats remixing as good, with credit.

COMMON WRONG ANSWERS & HOW TO COACH THEM

"Kids have to share to use Scratch." No: an account is optional, and projects can stay private forever.

Username like "EmmaJones2015". Real name + birth year. Suggest interest-based names (StarChef_77).

Rules that are only "don'ts." Add the positive rule: "If anyone's unkind, show me and we'll handle it together." That's the one that keeps kids talking to you.

5.3 · SCRATCHJR — CODING FOR AGES 5–7

Q1

How are blocks arranged in a ScratchJr script?

B) In a horizontal row, read left to right

Why: like a sentence. Scratch stacks top to bottom (A); order always matters (C).

Q2

React when your child touches the pet on screen?

C) Start on Tap

Why: Start on Bump fires when two characters touch; Green Flag starts when the flag is pressed.

Q3

Best session length for a 5-year-old?

A) About 10–15 minutes, ending while it's still fun

Why: stop after a big laugh and they'll ask to do it again tomorrow. Persistence isn't learned by pushing a five-year-old to tears.

Q4

The pet ends up bigger after every tap. Fix?

A) Add a Shrink block (same number) or Reset Size before End

Why: Grow changes size permanently; the script must undo it. From the lesson's common-problems table.

LAB — "MY PET SAYS HI": THE REFERENCE ROW

 Start on Tap

 Hop 2

 Grow 2

 My Sound

 Shrink 2

 End

Stretch (two-page story): page 1 — *Start on Green Flag* → *Say "Come to my house!"* → *Wait 10* → *Go to Page 2*; page 2 — *Start on Green Flag* → *Move Right 3* → *Say "Welcome home!"* → *End*. Go to Page only appears in the red End category once a second page exists.

Role-play look-fors: story asked for before any block · coach reads, "child" taps · hands in lap (steadyding the tablet is fine) · a laugh at the unexpected ("Whoa, that's not what we planned!") · ends on a win.

COMMON PROBLEMS & HOW TO COACH THEM

Tapping does nothing → script starts with Green Flag, or a block isn't snapped on. "Drag them until they click into one row."

Pet walks off the stage → moves add up each run; start the row with blue **Go Home**.

Silent recording → allow the microphone for ScratchJr in the tablet's settings.

A block "vanished" → it was dragged back onto the palette, which deletes it. Drag a fresh one out.

Color mix-ups ("green is Operators!") → that's Scratch. In ScratchJr green is Sound. Trust the pictures.

5.4 · WHAT COMES AFTER SCRATCH

Q1

Natural next thing to explore inside Scratch after Star Catcher?

B) Lists, or an extension like Pen or Music

Why: Scratch has years of depth; there's no rush to leave it (C).

Q2

Why is Scratch → Python not as scary as it sounds?

A) The core concepts transfer; only the syntax changes

Why: sequences, loops, conditionals, variables and events exist in every language. They're not identical (B), and nobody starts from zero (C).

Q3

Best way to keep a child's coding momentum?

B) Follow their interests with another project they're excited about

Why: interest sustains itself; pressure (A, C) burns it out.

Q4

Where are the Pen and Music blocks?

B) Behind the Add Extension button, bottom-left of the editor

Why: extensions add whole new block categories (Pen, Music, Text to Speech, Video Sensing, micro:bit). Scratch is free: there is no paid version (C).

LAB — OUR NEXT PROJECT: WHAT GOOD LOOKS LIKE

A named project the **child** is excited about (not the parent's plan), tied to one road-map branch. Examples: a high-score list for Star Catcher (deeper Scratch: lists) · a drawing toy (Pen extension) · a banana piano (Makey Makey) · a first Python loop (text code).

repeat 3

say Hi! for 1 seconds

```
for i in range(3):  
    print("Hi!")
```

COMMON WRONG ANSWERS & HOW TO COACH THEM

"They should start Python at 8 so they don't fall behind." Switch when curious, not on a schedule. Many kids start around 10–13; plenty stay in Scratch longer. Nothing is wasted.

"I can't keep up, so I'm done." Your job changes, it doesn't end: audience, question-asker, cheerleader.

WHAT A GOOD SHOWCASE LOOKS LIKE

The **child** presents and drives; the parent adds one sentence about a hands-off moment · every introduction praises process ("...who kept testing until the stars fell") · gallery-walk comments are kind, curious and specific · no photos without consent · every family leaves with a certificate, the "What Next" page, and one named next project.